

# **STOR 538 Playoffs Project**

Casey Hinckley

# Data Information

To create the passing yards model, two datasets were accessed through the R package *nflreader*:

1. Play by play data loaded from the nflverse data repository using the function *nflreadr::load\_pbp()*.
2. Game schedule data, including game identifiers, season, week, and the home and away team, loaded with *nflreadr::load\_schedules()*.

The initial tables were loaded and filtered to only include data for 2018 through the current season, as all but 7 current starting NFL QBs became a starter after 2018. Then rows without a possessing team were dropped, and only regular season games were kept to prevent different play styles, strategies, and other outside factors present in playoff games from skewing predictions.

A new variable `sack_yards_lost` was created that examined the yards lost on a play that resulted in a sack. This made it possible to subtract total sack yards from total pass yards to create another new variable `net_passing_yards` that would measure team passing yards in the manner reported by the NFL, which subtracts total yards lost on sacks from the total passing yards.

Play-by-play data was then summed by team and game, resulting in one row per team per game with aggregated game level totals such as:

- `sum_pass_attempt`
- `sum_net_passing_yards`

- sum\_sack
- sum\_rush\_attempt
- sum\_complete\_pass
- sum\_incomplete\_pass
- sum\_interception
- Sum\_pass\_touchdown
- sum\_sack\_yards\_lost

These rows were mapped to the schedule table, generating one row per game with columns for the home and away team values of each of the aggregated game level totals, and added columns defining the home and away teams for a given game.

After that, the table was pivoted to a long format table, generating two rows per aggregated game level total variable. The first containing a given team, game id, week, opponent, and the relevant game level total, and the second labeling the relevant game level total as *allowed\_metric\_name*. This would make it possible to easily sum rolling aggregates for offensive and defensive metrics, generating game level averages for both offense and defense.

For example, for a game between Arizona and Washington, there is one row for passing yards, and one row for allowed passing yards. One could then aggregate average passing yards and average passing yards allowed for each team over a given window and include both in a model.

# Methodology for Passing

To emulate “week 6 → predict weeks 7–11” without peeking, I built decayed window features frozen at anchor week  $t$  for each season:

- **Window:** Broke each of the selected previous NFL seasons into many 11 week windows defined by week  $t$ . For a given week  $t$ , the model used the data from weeks  $(t - 5, \dots, t)$  to predict passing yards for weeks  $(t + 1, \dots, t + 5)$ .
- **Decay:** Summarize each team’s current form with a six-week decay-weighted average that gives much more weight to the most recent games, then shrink that number toward the multi-year league average by adding 6 “ghost games” set to priors equal to the league average across the chosen time frame. This helps prevent early noise and byes from swinging the inputs.
  - $\text{Feature} = (n_0 \cdot \text{prior} + n \cdot \text{per\_game}) / (n_0 + n)$  where  $n_0 = 6$
  - This meant that for predicting any  $t + 1$  through  $t + 5$  week window, the closer a game was to week  $t$ , the higher its weight in the computed average.
  - A geometric decay of 0.6 was chosen because it roughly halves the influence of each week as distance increases, and it performed the best in the testing process.
- **Frozen features:** For each team I compute per-game, decayed metrics and their prior blends, then prefix them `s2d_*`. This includes net passing quantities (e.g., `s2d_sum_passing_yards_net`, `s2d_allow_sum_passing_yards_net`). All data for these frozen features come from data prior to week  $t$ .

I then constructed a matrix for each test window ( $t+1, \dots, t+5$ ) containing the relevant  $s2d\_*$  variables for the home and away team, appending  $home\_*$  or  $away\_*$  to each of the variables, and computed a benchmark for each window defined by:

- $\hat{Y} = home\_att * home\_ypa + away\_att * away\_ypa$

Where:

- $home\_ypa = home\_s2d\_sum\_passing\_yards\_net / home\_s2d\_sum\_pass\_attempt$
- $away\_ypa = away\_s2d\_sum\_passing\_yards\_net / away\_s2d\_sum\_pass\_attempt$

I also created similar variables for the allow values discussed earlier:

- $allow\_home\_ypa =$   
 $home\_s2d\_allow\_sum\_passing\_yards\_net / home\_s2d\_allow\_sum\_pass\_attempt$
- $allow\_away\_ypa =$   
 $away\_s2d\_allow\_sum\_passing\_yards\_net / away\_s2d\_allow\_sum\_pass\_attempt$

For each season  $s$  and each anchor week  $t$ :

1. Freeze inputs at week  $t$
2. Train the model for block  $(s, t)$  on all other blocks; the current block is held out.
3. Predict weeks  $(t+1, \dots, t+5)$
4. Choose a single  $\alpha$  per model family by minimizing mean MAE across all blocks (5 fold CV inside each fit).

Then fit three game-level linear models that all predict team net passing yards from the frozen week-6 features:

1. **Compact Elastic Net:** uses attempts, net YPA, opponent-allowed net YPA, gaps (offense minus opponent-allowed), and simple proxies (attempts×YPA, totals, and attempt differentials).
2. **Transformed Elastic Net:** same inputs plus light transformations (log/sqrt of attempts and (allowed) YPA) and a small set of football-meaningful interactions (attempts×own YPA, attempts×opponent-allowed YPA, gap×attempts).
3. **Polynomial Elastic Net (degree 2):** a quadratic expansion of the core compact features (all pairwise products and key squares like attempts<sup>2</sup> and YPA<sup>2</sup>) to capture curvature and interactions while staying in a linear framework.

I chose  $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$  to cover the full EN spectrum from ridge (0) to lasso (1) while including three balanced hybrids that capture the main behaviors. For each  $\alpha$  in this grid, I ran the full rolling backtest: in every (season, anchor) block the model trained on all other blocks, let 5-fold CV pick  $\lambda = \text{lambda.min}$  using MAE, and scored the held-out block. I then averaged MAE across all blocks and fixed the single global  $\alpha$  that minimized this average. The polynomial degree was fixed at 2. Finally, after scoring all held-out blocks, I compared each model family's average MAE to the frozen net-YPA baseline, with the degree-2 polynomial Elastic Net performing the best.

| Model<br><chr> | MAE<br><dbl> |
|----------------|--------------|
| Poly_EN        | 88.53878     |
| Ridge_Game_Tx  | 88.71943     |
| Ridge_Game     | 89.88750     |
| Baseline       | 90.78090     |

For the final model, I took the main variables from the base model and added their squares and simple pairwise products. Then squared home and away pass attempts, home and away net YPA, and home and away allowed net YPA. Finally, I also added products like home attempts times home YPA, home YPA times away allowed YPA, and other combinations of offensive and defensive features for both teams.

```
--- Final Polynomial Elastic Net Model ---
Chosen alpha: 0.75
Chosen lambda: 0.02765572
Number of nonzero coefficients: 49

Top coefficients:
      away_allow_ypa      away_ypa      home_allow_ypa
      299.283436      28.367850      25.705651
away_allow_ypa:home_ypa_gap      home_att      home_allow_ypa:away_ypa_gap
      24.759573      18.396205      13.810199
      home_ypa_gap:away_ypa_gap      home_ypa:home_ypa_gap      I(home_ypa^2)
      9.144955      7.679887      2.689298
      away_att:away_ypa
      1.366606
```

For my final predictions, I retrained the Poly\_EN model once on all the backtest data and applied it to the frozen Week 6 features of the target season to predict passing yards for Weeks 7 through 11.